

Supplementary Material for *SecAI* Paper

No Author Given

No Institute Given

Abstract. This document contains additional information for the paper *Supporting Developers in Secure Coding: Systematically Improving SAST Tool Usability Through AI-Enhanced Feedback*. This includes more detailed information on how the features presented in Section 3 of the aforementioned paper map to usability criteria. Furthermore, additional data is provided from the performance comparison reported in Section 4.2 of the original paper.

1 Mapping *SecAI* Features to Usability Criteria

A common problem with existing static analysis tools is the lack of usability. Previous research [4] has shown that most static analysis tools have severe usability shortcomings, leading to lower adoption rates and even abandonment of tools due to user dissatisfaction. A large-scale study of 46 static analysis tools based on 36 usability criteria showed that most of the tools lacked in terms of usability and improved user experience [4].

Nachtigall et al. [4] broadly defines six major usability categories. *Warning Messages* refers to the method in which warnings and errors are communicated to the user, whether they are integrated into the code, provide a detailed description of the error, severity and category information, etc. *Fix Support* includes quick fixes, providing alternative solutions, and helping the user understand how to fix the issue with the help of examples, previews, or tutorials. The *False Positives* section deals with features to improve false positive classification, such as a confidence factor that asks for the user’s knowledge to provide more useful false positive warnings and suppression of false positives if they are irrelevant to the user. *Usage of User Feedback* deals with allowing the user to customize and fine-tune the tool to their preferences by allowing suppression of certain warnings, customizing analysis rules, and filtering the warning alerts. *Workflow Integration* refers to whether the static analysis tool can be reasonably integrated into the user’s workflow without disrupting it, via IDE integration, standalone tools, browser access, etc. The final category, *Specialized UI*, evaluates how the tool interacts with the user, whether it makes use of highlighting warnings in code or using icons to display them, if the tool conforms to the general IDE/CLI design, offering customizability and similar features.

1.1 Feature Selection

Each feature of our *SecAI*-plugin was selected w.r.t. a specific focus area, as can be seen by the clustering of criteria in Table 1. *Exact Error Localization*, *Error*

Message and Description, and *Visualizing Interconnected Errors* are dedicated to warning messages and navigating between errors whilst often using specialized user interface components. Additionally, the latter, as well as *Severity Score* and *False Positive Detection*, were selected to cover warning prioritization while also making use of user feedback. On the other hand, the features of *AI-assisted Code Fix* and *Quick Fixes* assist with fix support. Outside of these more traditional categories *Secure Code Generation* simply improves the availability of secure code.

In addition to the individual features, Table 1 also contains the column *SonarQube [7] Plugin* to denote the usability criteria that the entire *SecAI*-plugin will fulfill on the basis of the features that the platform itself contains. This is because it does cover usability criteria that the actual features do not by nature of being a full program instead of individual components.

1.2 Comparing Planned and Actual Coverage of Usability Criteria

Table 1 maps the proposed *SecAI* features to the usability criteria that each feature was planned to cover (●) and actually meets (●). The fully filled circles denote a fulfilled criterion, partially filled circles denote a usability criterion which is partially met, and empty circles mark when a feature does not satisfy a usability criterion during planning or afterwards. The proposed *SecAI* tool was planned with the goal of fulfilling all 36 criteria, with many features covering multiple criteria. However, we were unable to meet three of the criteria entirely, namely *Fix Tutorial*, *Validation of True Positives*, and *Temporal Suppression*. Furthermore, not all issues are illustrated with a different example, meaning the *Explain Problem by Example* criterion is only partially met. Other discrepancies between planned and fulfilled criteria are elaborated on in the following sections.

Visualizing Interconnected Errors The initial plans for this feature revolved around displaying an error tree and allowing the user to click on a node to jump to the related warning. Although this is also the way it was in our custom *SecAI* view, attaching additional locations to the *SonarQube* [7] issues enabled us to fulfill additional criteria, specifically *Highlighting in Code* and *IDE/CLI Conformity*.

Quick Fixes Originally, it was planned to offer IDE conform quick fixes to users. However, even though *SonarQube* [7] offers quick fixes in their *SonarQube for IDE* plugin [8], this is only available for their own native rules and not for custom plugins or rules. Therefore, the quick fix information can only be displayed server-side, and immediate implementation through code actions is not possible.

AI-assisted Code Fix For this feature, we had difficulty classifying whether or not it would count as a quick fix even though the API request is not always

quick and whether dynamically generating the fix and therefore having the possibility of offering different solutions would fulfill the criterion of *Alternative Fix Solution*. In the end, it was decided to mark it as *Alternative Fix Solutions* but not *Quick Fix* since the latter was already covered by a separate feature.

False Positive Detection Due to the way that false positive detection had to be integrated, it was not possible to offer the intended search and filter options based on the computed confidence score. Furthermore, we could not add our own user-determined threshold-based approach due to the forementioned issue. This is also why the only way to suppress false positives is through *SonarQube*'s [7] native functionality, and is in no way related to our feature.

Table 1: Connecting features to the planned (left, ●) and actual (right, ●) fulfillment of usability criteria [4]. ○ denotes that coverage was not planned or did not succeed

Features		Exact Error Localization	Error Message and Description	Visualizing Interconnected Errors	Quick Fixes	AI-assisted Code Fix	Secure Code Generation	Severity Score	False Positive Detection	<i>SonarQube</i> [7] Plugin
Usability Criteria										
Warning Messages	Internal Reasoning	○/○	●/●	○/○	○/○	○/○	○/○	○/○	○/○	●/●
	Warning Connected to Code	○/○	●/●	○/○	○/○	○/○	○/○	○/○	○/○	●/●
	Clickable Trace	○/○	●/●	●/●	○/○	○/○	○/○	○/○	○/○	●/●
	Explain Consequences of Issues	○/○	●/●	○/○	○/○	○/○	○/○	○/○	○/○	●/●
	Offer More Information	○/○	●/●	○/○	○/○	○/○	○/○	○/○	○/○	●/●
	Explain Problem by Example	○/○	●/●	○/○	○/○	○/○	○/○	○/○	○/○	●/●
	Unique Identifier	○/○	●/●	○/○	○/○	○/○	○/○	○/○	○/○	●/●
	Description	○/○	●/●	○/○	○/○	○/○	○/○	○/○	○/○	●/●

Usability Criteria		Error Localization	Message and Description	Interconnected Errors	Quick Fixes	AI-assisted Fix	Code Generation	Severity Score	False Positives	<i>SonarQube</i> [7] Plugin
(Specialized) UI	Highlighting in Code	●/●	○/○	○/●	○/○	○/○	○/○	○/○	○/○	●/●
	Warning Icons	●/●	●/●	○/○	○/○	○/○	○/○	○/○	○/○	●/●
	IDE/CLI Conformity	●/●	●/●	○/●	●/○	○/○	○/○	●/●	○/○	●/●
	Customizability of the User Interface	○/○	○/○	●/●	○/○	○/○	○/○	○/○	○/○	●/●
	Overview of Warnings	○/○	●/●	●/●	○/○	○/○	○/○	○/○	○/○	●/●
	Search through Warnings	○/○	●/●	○/○	○/○	○/○	○/○	●/●	●/○	●/●
	Tracks Progress	○/○	○/○	○/○	○/○	○/○	○/○	○/○	○/○	●/●

2 Performance Comparison between *CogniCrypt* and *SecAI*

Table 2 contains all runs for the performance comparison *CogniCrypt* [3] run via CLI and through our *SecAI*-plugin. The analysis was executed five times each on relevant open source benchmarks, specifically *CryptoAPI-Bench* [1] and *Cam-Bench* [5]. All analysis runs were executed on the same machine (AMD Ryzen 5 5600G processor, 32 GB RAM, RTX 3060 (12 GB) graphics card, Windows 10 64-bit OS) on the same set of *CrySL* rules [2]. As the encapsulation of our plugin interfered with *CogniCrypt*'s ability to correctly parse the *CrySL* rule for `javax.servlet.http.Cookie`, it was excluded from all runs.

Another effect of the encapsulation is that the analysis has problems finding even basic Java classes such as `java.lang.Object` to resolve concrete methods. This is why the call graph constructed from within our *SecAI*-plugin has far fewer edges compared to the analysis via CLI (cf. Table 2) and can therefore be built more quickly. However, it should be noted that despite this the same number of errors was found during every run for both CLI and our *SecAI*-plugin.

3 Limitations

While using *SonarQube* [6] for our plugin has advantages as an established platform, including workflow integration for the existing developer user base, the

		<i>CamBench</i> - <i>Cap</i> [5]				CryptoAPI-Bench [1]			
		Analysis time	Edges in call graph	Call graph construction time	Typestate analysis time	Analysis time	Edges in call graph	Call graph construction time	Typestate analysis time
CLI	Run 1	7.130 s	15677	1.864 s	3.680 s	1.471 s	12404	1.477 s	0.766 s
	Run 2	7.150 s	15676	1.826 s	3.541 s	1.467 s	12399	1.510 s	0.791 s
	Run 3	6.867 s	15676	1.849 s	3.670 s	1.460 s	12398	1.464 s	0.737 s
	Run 4	6.687 s	15677	1.848 s	3.612 s	1.429 s	12399	1.481 s	0.733 s
	Run 5	6.735 s	15652	1.844 s	3.619 s	1.431 s	12422	1.524 s	0.726 s
	Average	6.914 s	15671	1.846 s	3.624 s	1.441 s	12404	1.491 s	0.750 s
<i>SecAI</i> -plugin	Run 1	9.678 s	5253	1.571 s	5.158 s	1.736 s	1523	0.519 s	0.956 s
	Run 2	9.293 s	5253	1.248 s	4.769 s	1.717 s	1523	0.549 s	0.950 s
	Run 3	9.058 s	5253	1.179 s	4.726 s	1.681 s	1523	0.514 s	0.923 s
	Run 4	9.383 s	5253	1.176 s	4.786 s	1.703 s	1523	0.512 s	0.880 s
	Run 5	9.621 s	5253	1.206 s	4.739 s	1.781 s	1523	0.493 s	0.906 s
	Average	9.407 s	5253	1.276 s	4.836 s	1.724 s	1523	0.517 s	0.923 s

Table 2. Performance comparison between running *CogniCrypt* [3] release 5.0.1 from the command line interface (CLI) and with our custom *SecAI*-plugin for two benchmarks.

native *SonarQube* [7] issue interface limits what information can be displayed to established API endpoints. Therefore, not all features can be directly integrated into the native interface. However, *SonarQube*’s plugin mechanism also includes the ability to add custom web pages, which we use to provide our custom *SecAI* view under the *More* tab for each project. However, due to limitations of the *SonarQube* plugin framework [6], we can only visualize these scores in the custom *SecAI* view. Our integration is thereby limited to only calculating the scores per issue when a user looks at it. Therefore, the planned sorting and filtering by *confidence* and *priority* does not work. While every feature was successfully developed and integrated, two of them were limited by our use of *SonarQube* [6] as the underlying platform. First, although *SonarQube* natively supports *Quick Fixes*, its plugin architecture does not permit the integration of custom quick fix functionality. Second, *False Positive Detection* faced structural limitations. Since priority scores are computed on demand after requesting the false positive score via API within the frontend, filtering by confidence or priority is not supported. Likewise, these scores cannot be embedded directly into the native *SonarQube* interface, as computing them during the analysis would take too much time, and are therefore only displayed in the *SecAI* custom view. These restrictions emphasize the trade-off between integrating into an established ecosystem that is already embedded in many workflows, and creating a new, standalone tool that offers maximum feature integration and fast response times.

References

1. Afrose, S., Rahaman, S., Yao, D.: Cryptoapi-bench: A comprehensive benchmark on java cryptographic api misuses. In: 2019 IEEE Cybersecurity Development (SecDev). pp. 49–61. IEEE (2019)
2. CrySl rules, <https://github.com/CROSSINGTUD/Crypto-API-Rules>, accessed: 2025-10-08
3. Krüger, S., Späth, J., Ali, K., Bodden, E., Mezini, M.: Crysl: An extensible approach to validating the correct usage of cryptographic apis. *IEEE Transactions on Software Engineering* **47**(11), 2382–2400 (2019). <https://doi.org/10.1109/TSE.2019.2948910>
4. Nachtigall, M., Schlichtig, M., Bodden, E.: A large-scale study of usability criteria addressed by static analysis tools. In: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis. p. 532–543. ISSTA 2022, Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3533767.3534374>, <https://doi.org/10.1145/3533767.3534374>
5. Schlichtig, M., Wickert, A.K., Krüger, S., Bodden, E., Mezini, M.: Cambench–cryptographic api misuse detection tool benchmark suite. *arXiv preprint arXiv:2204.06447* (2022)
6. SonarSource SA: Sonar Plugin API (2025), <https://docs.sonarsource.com/sonarqube/latest/extension-guide/developing-a-plugin/plugin-basics/>, accessed: 2025-10-08
7. SonarSource SA: SonarQube - Code Quality Tool (2025), <https://www.sonarsource.com/products/sonarqube/>, accessed: 2025-10-08
8. SonarSource SA: SonarQube for IDE (2025), <https://www.sonarsource.com/products/sonarlint/>